

Advances DataBase Management System

►Teacher: Saba Inam



Introduction to Advanced Data Models: Object- Relational Data Model

Overview

- ▶ In this lecture, we will explore advanced data models, focusing on the Object-Relational Data Model (ORDBMS). This model extends the traditional relational model by incorporating features from object-oriented programming, enhancing the way complex data is stored and managed.

What is Data Model?

- ▶ A data model defines how data is structured, stored, and manipulated in a database system.
- ▶ **Example:** Relational model stores data in tables with rows and columns.

Why do we need advanced data model?

- ▶ The traditional relational model (RDBMS) works well for structured data but struggles with:
- ▶ **Complex data types** (e.g., images, videos, geospatial data).
- ▶ **Real-world entities** (e.g., a "Student" having multiple "Courses" and "Projects").
- ▶ Limited support for inheritance, encapsulation, and relationships beyond foreign keys.

What is the Object-Relational Data Model (ORDBMS)?

- ▶ **Definition:** The ORDBMS combines features of both relational databases and object-oriented programming to handle complex data and relationships more efficiently.
- ▶ **Relational Model Features:** Tables, rows, columns, primary keys, foreign keys.
- ▶ **Object-Oriented Features:** Objects, classes, inheritance, encapsulation, user-defined types.

Key Concepts of ORDBMS (Simplified with Examples)

a. Objects and Classes:

1. **Object:** A real-world entity (e.g., a Student).
2. **Class:** A blueprint for objects. (e.g., Student class with attributes like Name, ID, Courses).

Key Concepts of ORDBMS (Simplified with Examples) cont...

► CREATE TYPE Student AS (
 StudentID INT,
 Name VARCHAR(100),
 DateOfBirth DATE
);

Key Concepts of ORDBMS (Simplified with Examples) cont...

b. User-Defined Types (UDTs): Allow creating complex data types beyond standard types like INT, VARCHAR.

► **Example:** A custom type for an address.

```
CREATE TYPE Address AS (  
  Street VARCHAR(100),  
  City VARCHAR(50),  
  Zipcode INT);
```

Key Concepts of ORDBMS (Simplified with Examples) cont...

c. Inheritance (Extending Types): Similar to sub-classing in object-oriented programming.

Types can inherit from other types → IS-A relationship.

► **Example:** A Graduate Student inherits from Student.

```
CREATE TYPE GraduateStudent  
UNDER Student (  
    ThesisTopic VARCHAR(100)  
);
```

Key Concepts of ORDBMS (Simplified with Examples) cont...

d. Encapsulation: Data and methods can be bundled together.

► **Example:** Defining a method to calculate age.

```
CREATE FUNCTION  
CalculateAge(DateOfBirth DATE)  
RETURNS INT AS  
$$ SELECT EXTRACT(YEAR FROM  
AGE(DateOfBirth)) $$  
LANGUAGE SQL;
```

Key Concepts of ORDBMS (Simplified with Examples) cont...

e. Complex Data Types (Nested Structures):

ORDBMS can store nested data.

- ▶ **Example:** A student with multiple phone numbers.

```
CREATE TABLE Student (  
  StudentID INT,  
  Name VARCHAR(100),  
  Phones TEXT[]  
);
```

Reference Types has-a relationship

- ▶ - References point to objects stored elsewhere (like pointers).
- ▶ - Avoids data duplication, improves consistency.
- ▶ Example:
- ▶ `CREATE TYPE Department AS (`
- ▶ `dname VARCHAR(20)`
- ▶ `);`
- ▶ `CREATE TABLE Employee (`
- ▶ `name VARCHAR(20),`
- ▶ `dept REF Department`
- ▶ `);`

Key Concepts of ORDBMS (Simplified with Examples) cont...

f. Collections (Arrays and Sets): Arrays, lists, and sets can be stored as attributes.

► **Example:** A student enrolled in multiple courses.

```
CREATE TABLE Student (  
  StudentID INT,  
  Name VARCHAR(100),  
  Courses TEXT[]  
);
```

Object-Relational vs. Relational Database: Key Differences

Features	Relational DB (RDBMS)	Object-Relational DB (ORDBMS)
Data Types	Simple types (int, varchar)	Complex types(UDTs, nested types)
Relationships	Foreign keys	Object and references
Extensibility	Limited	Highly extensible with UTDs
performance	Optimized for structured data	Optimized for complex data
Examples	MySQL, PostgreSQL (default mode)	PostgreSQL(extended mode),Oracle

Object-Relational Mapping (ORM)

- ▶ - Maps programming objects → database tables.
- ▶ - Each class → table, each object → row.
- ▶ - Simplifies database access.
- ▶ Examples: Hibernate, JPA, Django ORM

Benefits of ORDBMS

- ▶ **Supports Complex Data:** Handles multimedia, geospatial, and hierarchical data.
- ▶ **Code Reusability:** Reuse object definitions and methods.
- ▶ **Better Data Integrity:** Data is stored more closely to how it appears in the real world.

Large Object Types (LOBs)

- ▶ - Used for large data storage.
- ▶ - BLOB: Binary data (images, videos).
- ▶ - CLOB: Character data (text, documents).

Use Cases of ORDBMS

- ▶ Multimedia Databases: Storing videos, audio, and images.
- ▶ Geographical Information Systems (GIS): For maps and geospatial data.
- ▶ Scientific Data Management: Handling complex experimental data.
- ▶ E-commerce: Managing product catalogs with varied attributes.

Limitations of ORDBMS

- ▶ **Complexity:** More difficult to learn compared to traditional relational models.
- ▶ **Performance:** Might be slower for simple datasets due to overhead.
- ▶ **Limited Adoption:** Not all databases fully support ORDBMS features.

ORDBMS in Real-World Databases

- ▶ **PostgreSQL:** One of the most popular ORDBMS.
- ▶ **Oracle Database:** Provides advanced object-relational capabilities.
- ▶ **IBM Db2:** ORDBMS features for complex data handling.

Summary of Key Takeaways

- ▶ ORDBMS blends relational and object-oriented features for handling complex data.
- ▶ **Key features** include user-defined types, inheritance, encapsulation, and complex data structures.
- ▶ ORDBMS is useful in domains like multimedia, GIS, and scientific research.